

Reinforcement Learning for boxes' sorting on a physical treadmill

Coppola Riccardo (10773170), Parma Valentino (10780461), Pezzi Francesco (11093867)

Abstract

This work investigates the use of Reinforcement Learning (RL) to control an actuated treadmill for the separation of parcels. The treadmill is modelled as a grid of independently actuated cells, each able to adjust the velocity and steering angle of the parcels it carries, and the objective is to ensure that boxes exit the platform separated by a minimum required gap. A PPO agent is trained offline to learn this separation policy from a reward function that combines a sparse exit bonus, a dense spacing signal, and a collision penalty. Two further policies are derived: an energy-aware agent and a curriculum-learning agent, which is exposed to increasing parcel flow and a larger number of concurrent boxes. The base and energy-aware agents both achieve high singulation accuracy, while the energy-aware agent reduces the average actuation energy by approximately 16% at equal accuracy. The curriculum-learning agent maintains good separation performance under substantially heavier traffic. The results indicate that RL can deliver robust parcel separation and that auxiliary objectives such as energy efficiency and higher throughput can be incorporated efficiently.

1 Introduction

Reinforcement Learning (RL) has proven to be an exceptionally powerful method to train autonomous system to adapt to a complex environment. An agent interacts with the environment and, through a trial-and-error process, 'learns' how to behave in it, guided by rewards for successful actions and penalties for failures. In the following study, the focus is on the optimization of a dynamic package sorting platform. The overall kinematic framework and multi-cell simulation architecture of the treadmill environment are based on the sorting framework proposed by [1]. The agent is trained to control the boxes path, through an actuated treadmill, ensuring that they exit the platform with a minimum required gap between each. The study will first focus on the reward function design and logical implementation.

2 Environment Implementation

The environment consists of an actuated treadmill and the generation of parcels. The actuated zone of the treadmill is made of 25 square cells, each measuring 0.2 m per side as illustrated in [Figure 1](#). The actuators on each square can either increase or decrease the velocity of the cell, or rotate its direction. Physical limits are constrained to $[0.6, 2.2]$ m/s for the velocity and $[-45, +45]$ degrees for the angle. The nominal velocity of the treadmill is 1.9 m/s, meaning that before entering and when exiting the actuated zone, all boxes move at this speed. No acceleration has been modeled in this environment, so the boxes instantaneously take the velocity of the cell they are currently on. Two new boxes are generated every 0.75 seconds, up to a maximum of 10 boxes per episode, and their lateral size is randomly generated between 5 cm and 40 cm. Once all boxes have been generated and have exited the treadmill, the episode is considered finished and the environment is reset.

2.1 Learning Algorithm

RL have demonstrated to be robust enough to train an agent in this kind of environment. Specifically, a Proximal Policy Optimization (PPO) is implemented to train the policy offline before being deployed to handle online scenarios. PPO is an actor-critic reinforcement learning method widely used for its balance of sample efficiency and training stability. The architecture consists of two separate neural networks: a network that maps the environment state to specific actuation commands (the actor), and network that evaluates the overall quality of the current state using Generalized Advantage Estimation (the critic). By decoupling these functions, the training efficiency is massively improved, allowing the critic to provide step-by-step feedback on the actor’s decisions, stabilizing the learning process. Furthermore, an entropy loss term (*EntropyLossWeight=0.01*) is incorporated to encourage continuous exploration, preventing the agent from converging prematurely on suboptimal separation strategies.

The chosen neural network architecture receives a $[100 \times 1]$ observation vector containing the properties of the active parcels (occupancy status, x - y position, and size) mapped across the 25-cell grid. Based on this state formulation, the actor network outputs a continuous control vector dictating the next actuation state for each cell. During training, the agent interacts with the environment using its current policy, accumulates the step-by-step reward (defined in [section 3](#)) and updates the network weights and biases via the PPO objective function. Both the actor and critic networks share a similar deep architecture, utilizing three hidden layers composed of 512, 256, and 128 neurons, respectively.

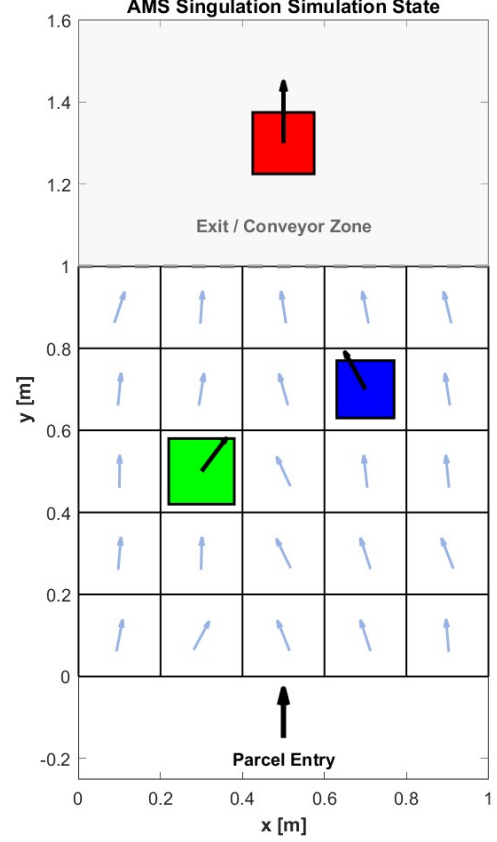


Figure 1: Treadmill AMS

3 Reward Function

3.1 Base Agent

First, a base agent was trained to separate parcels within the nominal environment previously described in [section 2](#). The reward function, detailed in [algorithm 1](#), operates on the following logic: The reward loop acquire the (x, y) position, velocity, and size of each active box on the treadmill, as well as the current actuation state of each platform cell. Processing the boxes in descending order of their y -position, the algorithm identifies the nearest successive parcel (measured strictly along the y -axis). The step reward is then calculated as follows: If a box successfully exits the platform with its nearest neighbor at or beyond the minimum required distance (Δy_{min}), a reward of +30 is given. If the successive box is too close, a penalty of -30 is applied. This heavily incentivizes the importance of the primary objective of successful singularization at the exit. However, to avoid a "last-minute brake" policy, where the agent aggressively slows all boxes just before the exit, a dense reward signal is introduced for boxes still in transit. Specifi-

cally, a reward or penalty of ± 1 is assigned per active box at each time step based on its current separation distance. This continuous feedback encourages the agent to initiate separating actions as early as possible, preparing the agent to handle scenarios with higher number of boxes, where last-minute braking would fail.

This dense evaluation is progressively guided by a spatial curriculum, where the active rewarding zone is initially restricted to the final section of the treadmill ($y \geq 0.8$) and linearly expands backward to the entrance ($y = 0.0$) over the first training episodes to stabilize early policy convergence.

Algorithm 1: Singulation Reward Function with Collision Penalty

Input: Active parcels P , Current episode E , Episodes to full belt E_{max}

Output: Total step reward R

```

1  $R \leftarrow 0$ ;
  /* Calculate expanding curriculum reward zone */
2  $y_{zone} \leftarrow 0.8 - 0.8 \cdot \min(1.0, E/E_{max})$ ;
3 for each parcel  $p_k \in P$  do
  /* Find the closest successive parcel behind  $p_k$  */
4    $\Delta Y \leftarrow \{y_k - y_j \mid \forall p_j \in P, y_k > y_j\}$ ;
5    $\Delta y \leftarrow \min(\Delta Y)$ ;
6    $p_{succ} \leftarrow$  parcel corresponding to  $\Delta y$ ;
  /* Calculate physical bounds and target thresholds */
7    $\Delta_{collide} \leftarrow \frac{d_k + d_{succ}}{2}$ ; /* Physical boundary */
8    $\Delta y_{min} \leftarrow \Delta_{collide} + \text{shift}$ ; /* Singulation boundary */
9    $\Delta x \leftarrow |x_k - x_{succ}|$ ;
  /* Physical Collision Penalty (-5) */
10  if  $\Delta y < \Delta_{collide}$  and  $\Delta x < \Delta_{collide}$  then
11     $R \leftarrow R - 5$ ;
12  end
  /* Evaluate Spatial Curriculum Zone (+1 / -1) */
13  if  $y_k \geq y_{zone}$  and  $y_k < 1.0$  then
14    if  $\Delta y < \Delta y_{min}$  then
15       $R \leftarrow R - 1$ ; /* Spacing penalty */
16    else
17       $R \leftarrow R + 1$ ; /* Spacing reward */
18    end
19  end
  /* Evaluate Exit Zone (+30 / -30) */
20  if  $p_k$  is crossing the exit line then
21    if  $\Delta y \geq \Delta y_{min}$  then
22       $R \leftarrow R + 30$ ; /* Successful singulation bonus */
23    else
24       $R \leftarrow R - 30$ ; /* Failed singulation penalty */
25    end
26  end
27 end
28 return  $R$ ;

```

3.2 Energy-aware agent

RL policies could result on highly aggressive control trajectories. It is important to remember that this algorithm will ultimately be deployed on a real physical system where strict hardware constraints and energy limitations are to be considered. This section details the implementation of the energy-aware agent, designed to minimize the actuation energy of the treadmill without compromising the sorting accuracy. The trained base agent is utilized as the starting point, as the reward maintain the core sorting strategy, but introduces an additional term that penalize excessive energy utilization. The energy per time-step is computed as described in Equation 1 and is decoupled in two distinct physical phenomena. The first term captures the dynamic power consumption required to transition between successive actuation states. The second term accounts for the steady-state power consumption, which scales with the rotational velocity of each individual actuator. This penalizes the continuous baseline energy lost to mechanical friction and electrical resistance, discouraging the agent from running the treadmill at unnecessarily high speeds. Due to the unavailability of specific physical parameters, such as the masses of the components and the dynamic friction factors, a strict physical derivation of power or energy consumption could not be implemented. Instead, a normalization approach was adopted for the three kinematic terms. Finally, the energy is scaled into a bounded reward penalty as follows: $R = -k(2 \cdot E_t/E_{t_{max}} - 1)$.

$$E_t = \frac{1}{\Delta\theta_{max}} \sum_{i=1}^{25} |\theta_{i,t} - \theta_{i,t-1}| + \frac{1}{v_{max}} \sum_{i=1}^{25} v_{i,t} + \frac{1}{\Delta v_{max}} \sum_{i=1}^{25} |v_{i,t} - v_{i,t-1}| \quad (1)$$

Where:

- $\Delta\theta_{max}$ represents the maximum allowable change in the angular orientation of the individual treadmill blocks.
- v_{max} denotes the upper velocity bound for a single treadmill block.
- Δv_{max} specifies the maximum permissible velocity variation (acceleration constraint) of an individual treadmill block.

3.3 Curriculum-Learning Agent

While the base agent manages parcels under the nominal load of two boxes generated every 0.75 s up to a maximum of ten per episode, a real line must cope with denser and more variable parcel flows. The third agent therefore targets to preserve the singulation accuracy while the environment is progressively driven toward higher arrival rates and a larger number of simultaneously active parcels.

This agent does not introduce any new reward term. The singulation reward of algorithm 1 is retained unchanged, but the difficulty of the environment is escalated over time. As with the energy-aware agent, the already-trained base policy is used as the starting point. The learning rate of both actor and critic is reduced by an order of magnitude (from 10^{-4} to 2×10^{-5}) to fine-tune the inherited policy without erasing the competence already acquired. The added changes in the environment are the following:

- The parcel spawn interval T_{spawn} is contracted from 0.70 s toward a hard floor of 0.40 s, increasing the temporal density of arrivals:

$$T_{spawn}(E) = \max\left(0.40, 0.70 - 0.30 \cdot \min\left(1, \frac{E}{E_{ramp}}\right)\right), \quad E_{ramp} = 1800. \quad (2)$$

- Second, the maximum number of concurrent parcels is raised from 10 to 15, allowing a longer queue of boxes to coexist on the platform and forcing the policy to plan separations further in advance

The ramp completes near episode 1800 out of the 2000 total number, so that the temporal spacing of arrivals is the dominant driver of difficulty: once the agent can separate parcels arriving every 0.40 s, it can comfortably handle the same number of parcels arriving more slowly.

4 Learning Process

4.1 Spatial Curriculum Learning

To prevent the agent from being overwhelmed early in training, the reward zone is progressively expanded. During the initial episodes, the ± 1 transit reward is restricted strictly to the exit of the treadmill, forcing the agent to first discover the primary objective of exit separation. As training progresses, this active reward zone gradually expands backward from the exit toward the entry point. This spatial curriculum approach significantly accelerates the learning process and yields more stable, robust policies. It takes 2000 episodes for the agent to reach the full observation zone. The spatial curriculum learning is used for the base agent only, successive agents will start with the knowledge of how to use space properly.

4.2 Curriculum Learning

Additionally, a curriculum learning strategy is implemented to progressively scale the environment’s difficulty. The already-trained base agent is loaded into a dynamic environment where both the parcel spawning frequency and the maximum number of concurrent boxes gradually increase. Because the agent has already learned to utilize the space properly, spatial restrictions are removed, allowing it to observe and operate across the entire treadmill.

5 Training & Results

5.1 Agents Training

The evolution of the episode reward during training is reported in Figures 2, 3 and 4 for the three agents. The base agent is trained from scratch, starting from a randomly initialised policy: in the earliest episodes it has not yet discovered the terminal singulation reward, so failed separations and collisions dominate and the cumulative reward is negative. As training proceeds the curve rises steeply once the agent discovers the +30 exit bonus. The energy-aware and curriculum-learning agents, by contrast, are not trained from scratch but fine-tuned from the already-competent base policy. They begin in positive-reward territory from the first episode, since the inherited policy already separates parcels reliably and collects mostly positive rewards. Their training curves consequently show no initial negative phase; they instead exhibit a smaller, more gradual adjustment as each agent adapts to its specific additional objective.

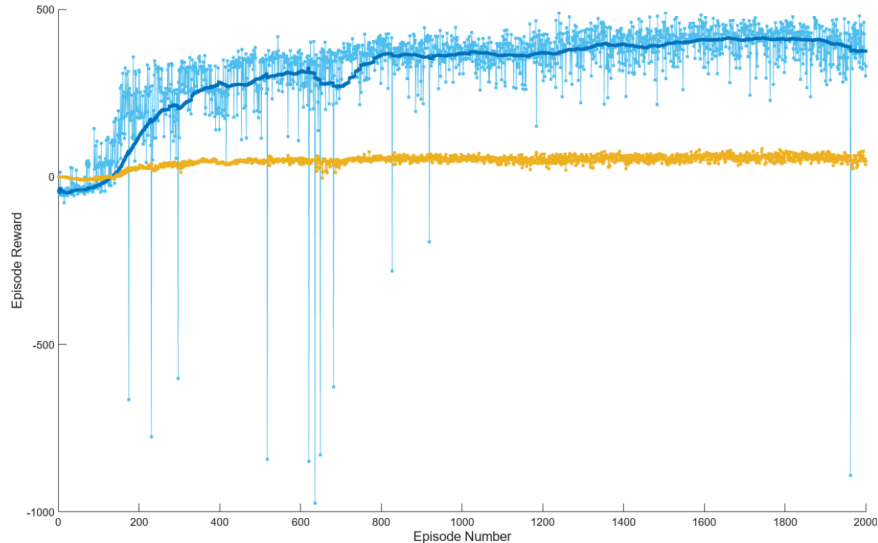


Figure 2: Base Agent Reward during Training

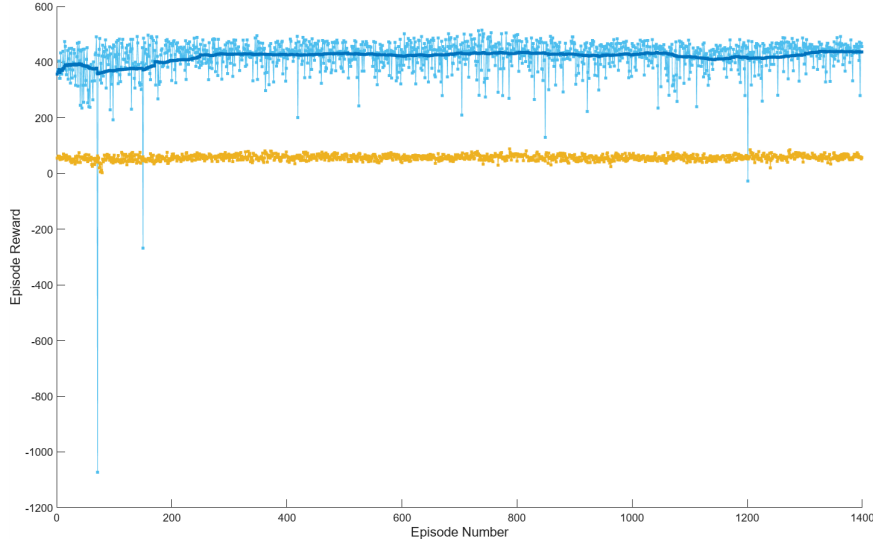


Figure 3: Energy-Aware Agent Reward during Training

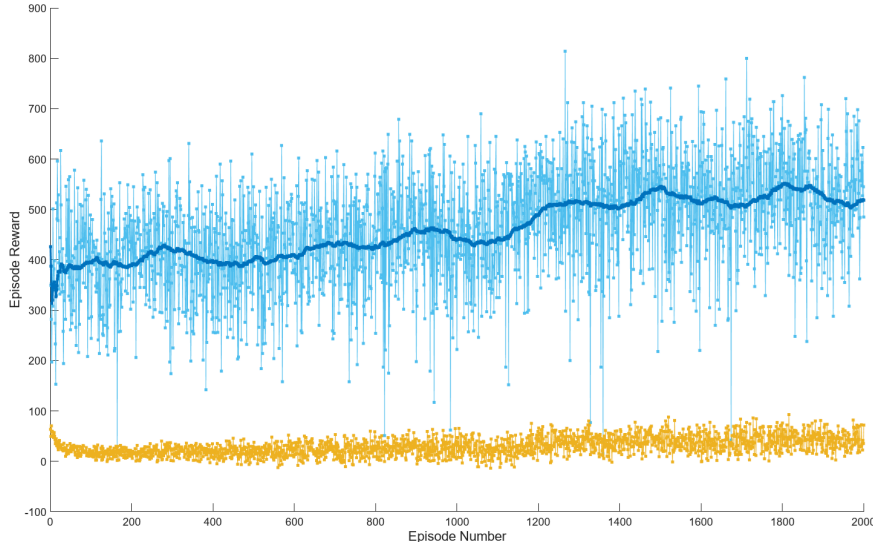
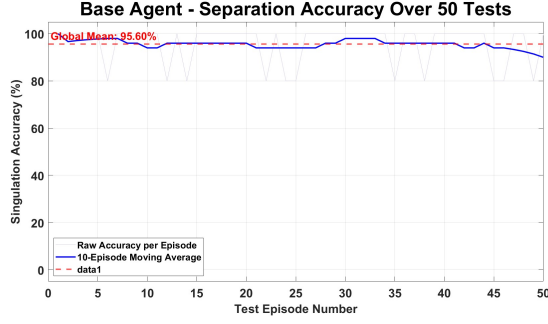


Figure 4: Curriculum-Learning Agent Reward during Training

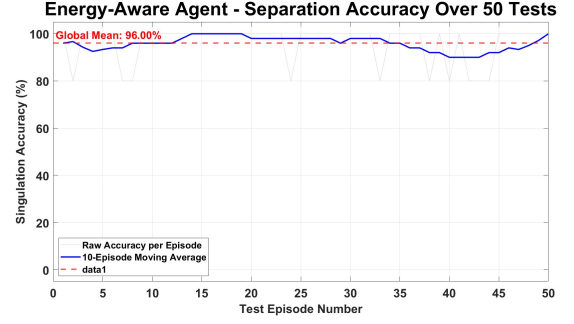
5.2 Agents Precision

The separation accuracy achieved by each agent over the independent test simulations is summarised in Figure 5. The base and energy-aware agents both attain a very high mean separation accuracy ($\sim 96\%$): the dense, well-separated reward function allows the policy to converge to a near-optimal singulation strategy. Notably, the energy-aware agent retains essentially the same accuracy as the base agent, confirming that the added energy penalty does not compromise the primary sorting objective.

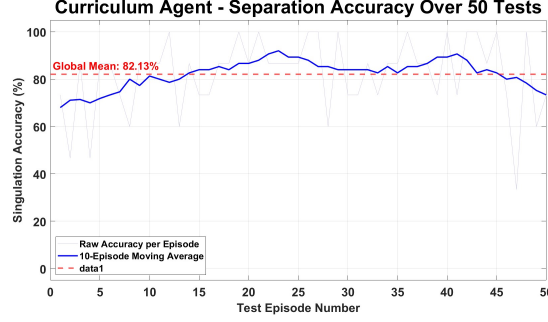
The curriculum-learning agent reports a lower, but still satisfactory mean accuracy ($\sim 82\%$). This is expected and does not indicate a weaker policy: unlike the first two agents, it is evaluated on a substantially harder task, with parcels arriving at a higher frequency and a larger number of boxes coexisting on the treadmill at once. Under these conditions occasional separation failures become unavoidable, which lowers the average score. Hence, the achieved accuracy remains good: the agent generalises the separation behaviour learned on the nominal load to a much more demanding throughput regime, without collapsing.



(a) Base Agent



(b) Energy-Aware Agent



(c) Curriculum Learning

Figure 5: Testing results and separation accuracy across all trained agents.

5.3 Agents Energy Consumption

The actuation energy consumed by the base and energy-aware agents over the 1000 test episodes is compared in Figure 6. The per-episode energy of the energy-aware agent lies consistently below that of the base agent for the testing episodes, indicating that the energy saving is systematic and not the result of a few favourable runs. Second, the overall mean energy is likewise lower for the energy-aware agent, with a reduction of approximately 16% relative to the base agent. Crucially, this saving is achieved without degrading the separation accuracy discussed previously, confirming that the energy term successfully steers the policy toward smoother, less aggressive actuation while preserving the primary sorting objective.

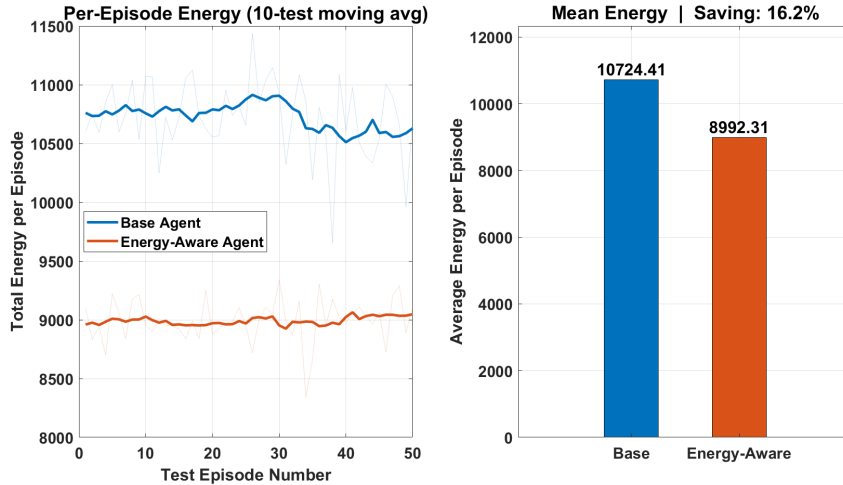


Figure 6: Base and Energy-Aware Agents Consumptions

6 Conclusions & Comments

In conclusion, the RL algorithm implemented demonstrated to successfully reach the accuracy for the sorting platform. The agent managed to learn from the complex environment to maximize the obtained reward and perform the required task. A critical thought emerges from the implementation of such a system in reality. The 'eyes' of the agent could be implemented through sensors on the treadmill, thus introducing measurement errors that could reduce the sorting accuracy. Another solution could be the use of an integrated camera above the sorting zone. Then the image could be processed through Convolutional Neural Networks (CNNs) and given as the observation input.

References

- [1] Loris Roveda, Marco Maccarini, Filippo Pura, Fabio Reiso, and Blerina Spahiu. Optimizing parcels sorting through reinforcement learning for intralogistics. In *Proceedings of the 27th European Conference on Artificial Intelligence (ECAI 2025)*, volume 342 of *Frontiers in Artificial Intelligence and Applications*, pages 5439–5446. IOS Press, 2025.